

Documentação de Integração Técnica: Serving API e Feature Store

Guia de Referência para equipes de Backend e MLOps

Engenharia de Dados e MLOps

23 de junho de 2026

Índice

1 Introdução

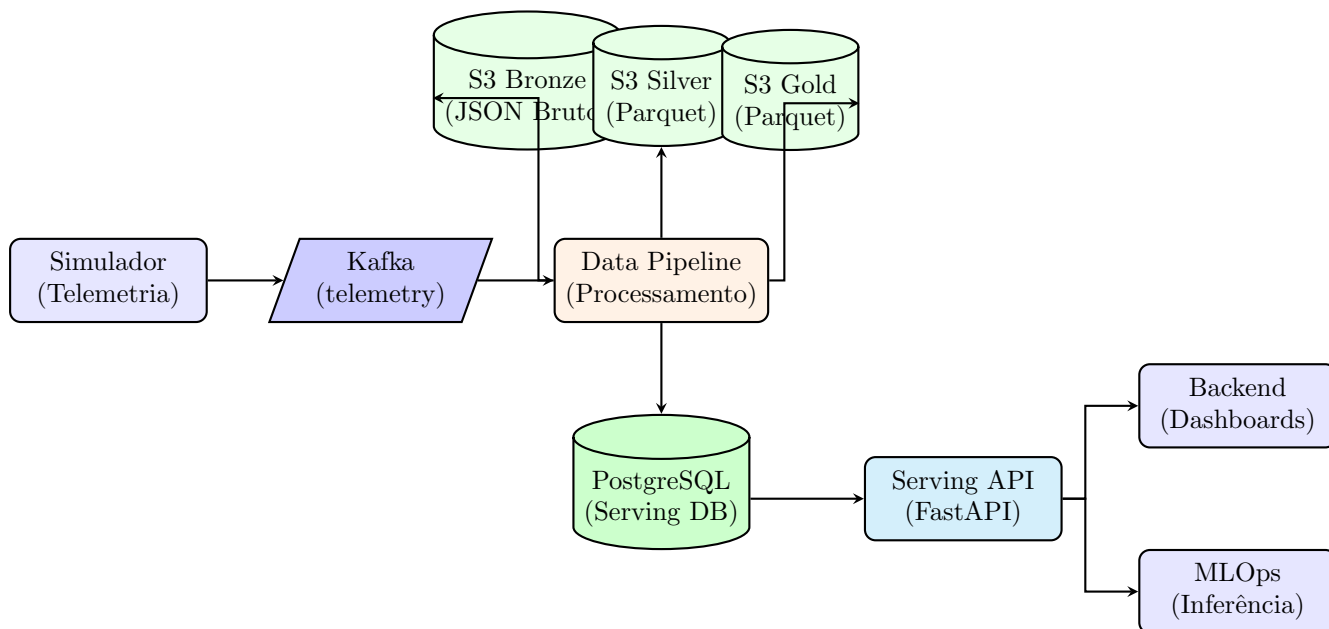
Este documento estabelece as especificações técnicas para a integração de sistemas com o pipeline de dados de telemetria hospitalar em tempo real e a sua respectiva **Serving API**. O sistema foi arquitetado para receber sinais de sensores de equipamentos médicos (tais como Tomógrafos, Aparelhos de Raio X, Ressonâncias Magnéticas, PET-Scans, Ultrassons e Arcos Cirúrgicos), submetê-los a um rigoroso tratamento de qualidade de dados (*Data Quality*) e transformações estatísticas causais, e disponibilizá-los em baixa latência.

O objetivo deste manual é orientar:

1. A equipe de **Backend (Desenvolvimento de Dashboards/Visualização)** no consumo das séries temporais tratadas na camada **Silver**.
2. A equipe de **MLOps (Engenharia de Machine Learning)** no consumo do vetor de features agregadas na camada **Gold** para alimentar modelos de predição de falhas online.

2 Arquitetura de Dados Geral

A arquitetura baseia-se no padrão **Medallion** combinado a um banco de dados relacional de *serving* de alto desempenho e uma API RESTful baseada em FastAPI. O fluxo de dados opera conforme ilustrado no diagrama a seguir:



- **Data Lake (MinIO/S3):** Atua como repositório analítico de persistência de longo prazo em formato colunar Apache Parquet.
- **Serving DB (PostgreSQL):** Armazena os dados mais recentes das camadas Silver e Gold, otimizado para consultas pontuais de baixa latência solicitadas pela API.

- **FastAPI Serving API:** Isola o banco de dados e espõe endpoints REST padronizados para consumo externo pelas equipes de Backend e MLOps.

3 Tratamento e Processamento de Dados

3.1 Camada Bronze: Ingestão Bruta

A camada Bronze realiza o arquivamento persistente do sinal original emitido pelos equipamentos. Cada payload de telemetria recebido do Kafka é gravado no MinIO como JSON bruto sem alterações. O particionamento é cronológico:

```
bronze-telemetry/hospital_id={id}/year={yyyy}/month={mm}/day={dd}/
```

3.2 Camada Silver: Limpeza e Qualidade de Dados

A camada Silver estrutura a série histórica de cada ativo sob regras rígidas de integridade física e temporal. O pipeline executa as seguintes etapas:

1. **Reindexação Temporal:** Cria uma grade horária uniforme de registros com passo fixo de 1 hora ($t_i = t_{i-1} + 1$ hora). Dados ausentes decorrentes de falhas de comunicação temporárias são inseridos como linhas de valores nulos.
2. **Interpolação Linear:** Se a lacuna de tempo sem sinal (*gap*) for menor ou igual a 2 horas ($L \leq 2$), os sensores são interpolados linearmente:

$$x(t) = x(t_0) + (t - t_0) \cdot \frac{x(t_1) - x(t_0)}{t_1 - t_0} \quad (1)$$

Onde t_0 é o último timestamp válido anterior ao gap e t_1 é o primeiro timestamp válido posterior. Se o gap for superior a 2 horas, os dados permanecem como NULL (ou NaN no Pandas) para representar inatividade ou parada programada do equipamento.

3. **Clamping Físico:** Para impedir ruídos ou leituras impossíveis dos sensores, o pipeline limita as variáveis operacionais a seus limites de segurança definidos em regulamentações técnicas:

$$x_{\text{clamped}} = \max(x_{\min}, \min(x_{\max}, x)) \quad (2)$$

4. **Sinalização de Auditoria:** A coluna `is_interpolated` é atualizada para True nas linhas geradas artificialmente pela interpolação, permitindo que consumidores auditem ou filtrem essas leituras.

3.3 Camada Gold: Engenharia de Features de MLOps

A camada Gold lê os dados tratados da Silver e gera indicadores estatísticos para modelos preditivos. O pipeline aplica agregações baseadas em janelas deslizantes (*rolling windows*) causais de **6h, 12h, 24h e 72h**:

- **Métricas:** Média (`mean`), Desvio Padrão (`std`), Mínimo (`min`) e Máximo (`max`).
- **Prevenção de Data Leakage:** O cálculo é puramente retrospectivo (*lookback*), o que significa que o cálculo no timestamp t depende unicamente de observações no intervalo $[t - \text{janela}, t]$, impedindo que dados do futuro vazem para os conjuntos de treino e inferência.

4 Modelagem do Banco de Dados de Serving

O PostgreSQL serve como a camada de dados de acesso rápido para a API. A estrutura lógica das duas tabelas principais é descrita a seguir:

4.1 Tabela Física `stg_silver_telemetry` e View `silver_telemetry`

Armazena a série histórica confluída e limpa dos sensores. É uma tabela única e unificada de sensores. Colunas não aplicáveis a um tipo de equipamento permanecem como `NULL`. Para garantir compatibilidade com o Backend sem a necessidade de alterar consultas e schemas existentes, os dados são persistidos na tabela física `stg_silver_telemetry` e expostos através da view relacional `silver_telemetry`.

Tabela 1: Esquema da Tabela `stg_silver_telemetry`

Nome da Coluna	Tipo de Dado	Descrição
<code>timestamp</code> (PK)	<code>TIMESTAMP</code>	Registro temporal da leitura (grade horária regular).
<code>equipamento_id</code> (PK)	<code>INTEGER</code>	Código identificador do equipamento.
<code>hospital_id</code>	<code>INTEGER</code>	Código identificador do hospital.
<code>tipo</code>	<code>VARCHAR(50)</code>	Tipo de equipamento (ex: <code>tc</code> , <code>rx</code> , <code>rm</code> , <code>pet</code>).
<code>is_interpolated</code>	<code>BOOLEAN</code>	Flag indicativo se a linha foi interpolada.
<code>tube_temp</code>	<code>DOUBLE PRECISION</code>	Temperatura da ampola (TC/Raio X) - Nullable.
<code>tube_current</code>	<code>DOUBLE PRECISION</code>	Corrente do tubo (TC) - Nullable.
<code>anode_rotation_speed</code>	<code>DOUBLE PRECISION</code>	Rotação do ânodo (TC) - Nullable.
<code>detector_temp_drift</code>	<code>DOUBLE PRECISION</code>	Desvio da temp. do detector (TC) - Nullable.
<code>slip_ring_error_rate</code>	<code>DOUBLE PRECISION</code>	Taxa de erro do slip ring (TC) - Nullable.
<code>gantry_vibration_fft</code>	<code>DOUBLE PRECISION</code>	Vibração da gantry (TC) - Nullable.
<code>helium_level</code>	<code>DOUBLE PRECISION</code>	Nível de Hélio líquido (RM) - Nullable.
<code>transducer_temp</code>	<code>DOUBLE PRECISION</code>	Temperatura do transdutor (US) - Nullable.

4.2 Tabela `gold_equipment_features`

Armazena a feature store de MLOps. As variáveis calculadas em janelas móveis são consolidadas em uma única coluna `JSONB` para otimização e flexibilidade de schema (suporta mais de 130 colunas dinâmicas sem necessidade de alteração de DDL física no banco).

Tabela 2: Esquema da Tabela `gold_equipment_features`

Nome da Coluna	Tipo de Dado	Descrição
<code>timestamp</code> (PK)	<code>TIMESTAMP</code>	Timestamp associado ao fechamento da janela.
<code>equipamento_id</code> (PK)	<code>INTEGER</code>	Código identificador do equipamento.
<code>is_interpolated</code>	<code>BOOLEAN</code>	Indica se a leitura base é interpolada.
<code>features</code>	<code>JSONB</code>	Dicionário contendo as 130+ features de MLOps.

5 Especificação da Serving API

A Serving API está hospedada no ambiente Kubernetes e é acessada em:

`http://localhost:8000/api/v1`

5.1 Endpoint: Liveness/Readiness Probe

* **Método:** GET * **Rota:** /health * **Descrição:** Verifica a integridade da API e a conectividade com o banco de dados PostgreSQL. * **Retorno de Sucesso (HTTP 200 OK):**

```
1 {  
2   "status": "healthy",  
3   "database": "connected"  
4 }
```

5.2 Endpoints: Histórico de Telemetrias Silver

* **Consulta por Equipamento:**

- **Método:** GET
- **Rota:** /equipments/{id}/telemetry
- **Query Params:** limit (inteiro, opcional, padrão = 48)

* **Consulta por Hospital (Multi-Hospital):**

- **Método:** GET
- **Rota:** /hospitals/{hospital_id}/telemetry
- **Query Params:** limit (inteiro, opcional, padrão = 100)

* **Descrição:** Retorna a série histórica confluída para o(s) equipamento(s) solicitado(s), ordenada de forma decrescente por timestamp. Colunas de sensores cujo valor é nulo para o tipo de equipamento consultado são omitidas da resposta JSON final para manter a eficiência do payload de rede. * **Exemplo de Payload de Retorno (TC - Tomógrafo):**

```
1 [  
2   {  
3     "timestamp": "2026-06-16T21:33:01.450887",  
4     "equipamento_id": 1,  
5     "hospital_id": 1,  
6     "tipo": "tc",  
7     "is_interpolated": false,  
8     "tube_temp": 105.0,  
9     "tube_current": 349.341,  
10    "anode_rotation_speed": 9760.093,  
11    "detector_temp_drift": 3.676,  
12    "slip_ring_error_rate": 0.003,  
13    "gantry_vibration_fft": 0.274,  
14    "gantry_rotation": 0.394  
15  }  
16 ]
```

5.3 Endpoints: Últimas Features Gold

* Consulta por Equipamento (Online):

- Método: GET
- Rota: /equipments/{id}/features
- Descrição: Retorna as features mais recentes para predições de um ativo específico.

* Consulta por Hospital (Batch / Lote):

- Método: GET
- Rota: /hospitals/{hospital_id}/features
- Descrição: Retorna as features mais recentes de todos os equipamentos do hospital informado.

* Exemplo de Payload de Retorno (Consulta por Equipamento):

```
1 {  
2   "timestamp": "2026-06-16T21:33:01.450887",  
3   "equipamento_id": 1,  
4   "is_interpolated": false,  
5   "features": {  
6     "tube_temp_mean_6h": 42.92,  
7     "tube_temp_std_6h": 0.54,  
8     "tube_temp_max_72h": 68.21,  
9     "gantry_vibration_fft_mean_24h": 0.31,  
10    "gantry_vibration_fft_std_24h": 0.005,  
11    "slip_ring_error_rate_mean_12h": 0.003  
12  }  
13 }
```

6 Manual de Integração para a Equipe de Backend

O backend consome a rota de telemetria para desenhar gráficos e gauges nos painéis de monitoramento do hospital.

6.1 Diretrizes de Visualização no Dashboard (Frontend)

1. **Indicação de Dados Estimados (Interpolação):** Quando o campo `is_interpolated` for `True`, o frontend deve destacar visualmente o ponto correspondente na curva temporal (usando linhas tracejadas, cor de linha diferente, ou um ícone informativo no tooltip do ponto). Isso evita que os operadores tomem ações baseadas em leituras artificiais de segurança sem saberem do processamento de dados.
2. **Visualização de Inatividade (Downtime):** Gaps superiores a 2 horas não são interpolados, resultando em dados ausentes na série cronológica horária (não haverá registro ou os campos estarão ausentes/nulos). O frontend deve quebrar a linha gráfica nestes pontos, deixando um **espaço vazio** que representa inequivocamente que o equipamento esteve *offline* ou desligado.

6.2 Exemplo de Integração Frontend (JavaScript Fetch)

```
1 async function carregarDadosGrafico(equipamentoId) {
2   const url = 'http://localhost:8000/api/v1/equipments/${equipamentoId}/
   telemetry?limit=48';
3   try {
4     const response = await fetch(url);
5     if (!response.ok) throw new Error("Erro ao consultar Serving API.");
6
7     const dados = await response.json();
8
9     // Processamento para biblioteca de graficos (ex: Chart.js)
10    const timestamps = dados.map(d => new Date(d.timestamp).
      toLocaleString("pt-BR"));
11    const temps = dados.map(d => d.tube_temp);
12    const flagsInterpolacao = dados.map(d => d.is_interpolated);
13
14    desenharGrafico(timestamps, temps, flagsInterpolacao);
15  } catch (erro) {
16    console.error("Falha na integracao com a telemetria:", erro);
17  }
18 }
```

Listing 1: Implementação no Dashboard Client

7 Manual de Integração para a Equipe de MLOps

A equipe de MLOps utiliza o endpoint de features para obter o estado atual de variáveis estatísticas e passá-las para os pipelines de inferência de falhas em tempo de execução.

7.1 Alimentando o Modelo de Machine Learning

Como as features de treino foram geradas a partir de agregação retrospectiva e o endpoint expõe exatamente o mesmo dicionário de forma achatada, a equipe de MLOps pode carregar o payload de resposta da API de forma instantânea em um DataFrame do Pandas e passá-lo ao estimador de classificação.

7.2 Exemplo de Script de Inferência Online (Python)

```
1 import requests
2 import pandas as pd
3 import joblib
4
5 # Carrega o modelo de classificacao treinado (pre-requisito)
6 modelo = joblib.load("model_predictive_maintenance.pkl")
7
8 def predizer_falha_equipamento(equipamento_id):
9     url = f"http://localhost:8000/api/v1/equipments/{equipamento_id}/features"
10    try:
11        response = requests.get(url)
12        if response.status_code == 404:
13            print(f"Sem features calculadas para o equipamento {equipamento_id}")
14            return None
15        response.raise_for_status()
16
17        data = response.json()
18
19        # Converte o dicionario de features em um DataFrame de 1 linha
20        df_features = pd.DataFrame([data["features"]])
21
22        # O DataFrame ja contem todas as 130+ colunas no formato de treino:
23        # ex: 'tube_temp_mean_6h', 'gantry_vibration_fft_std_24h', etc.
24        probabilidade = modelo.predict_proba(df_features)[0][1]
25
26        return {
27            "timestamp": data["timestamp"],
28            "equipamento_id": data["equipamento_id"],
29            "is_interpolated": data["is_interpolated"],
30            "probabilidade_falha": probabilidade
31        }
32    except Exception as e:
33        print(f"Falha na inferencia online para o ativo {equipamento_id}: {e}")
```

34

```
return None
```

Listing 2: Consumo e Inferência Online com Serving API